



湖南大学
HUNAN UNIVERSITY

第三章 摊还分析

湖南大学计算机学院

目录

第一节

Java里面ArrayList例子引入

第二节

什么是摊还分析?

第三节

摊还分析的三种方法

Java ArrayList 的介绍



- 如何用数组实现链表?
- 典型实现就是Java里面的ArrayList, ArrayList数据结构本质: 基于连续内存的动态数组。

Java ArrayList 核心挑战



- ❑ 核心挑战：分配多少空间？
- ❑ 容量过小
 - ❑ 后果：频繁触发“数组已满”状态。
 - ❑ 代价：需要极高频率地分配新内存并复制数据，导致性能急剧下降。
- ❑ 容量过大
 - ❑ 后果：大部分内存单元长期闲置。
 - ❑ 代价：造成严重的空间浪费。

Java ArrayList的动态扩容机制



□ 使用Java ArrayList, 当表满时插入元素会产生溢出, 此时:

- ① 重新一个1.5倍大的表
- ② 将旧表里的元素复制到新表
- ③ 释放旧表空间

□ 关键参数

- ① 初始容量: 默认构造函数 `new ArrayList()` 不立即分配空间; 首次插入时, 自动扩容至默认值: $C_0 = 10$ 。
- ② 扩容策略: 当元素个数 `size ==` 当前容量 `capacity` 时; 新容量 $\approx$ 旧容量 $\times 1.5$ 下取整。

代价计算



有一个序列包括n个插入运算，在最坏情况下一次插入操作的代价为多少

$O(n)$

n个项的插入操作代价仍为 $O(n)$ ，分析如下：

设 c_i 为第i个插入的代价，则

$$c_i = \begin{cases} i & \text{若 } i-1 \text{ 恰为 } \lfloor 10 \times 1.5^k \rfloor \\ 1 & \text{其他} \end{cases}$$

代价计算



i	1	2	3		10	11	12		15	16
SIZE _i	10	10	10		10	15	15		15	22
C _i	1	1	1		1	11	1		1	16

因此, n 个插入运算的总代价为 $\sum_{i=1}^n c_i \leq n + \sum_{j=0}^{\lfloor \log_{1.5}(\frac{n}{10}) \rfloor} 10 \times 1.5^j$

$$< n + 10 \times \frac{(1 - 1.5^{\lfloor \log_{1.5} \frac{n}{10} \rfloor})}{(1 - 1.5)} = n + 20 \times (\frac{n}{10} - 1) = 3n - 20$$

每个插入运算的平均代价为 $O(n)/n$, 也就是 $O(1)$ 。

这就是摊还分析的思想, 这里采用了聚合分析方法

动态表 (17.4节)



□ 基于2倍的扩容，当表满时插入元素会产生溢出，此时：

- ① 重新一个2倍大的表
- ② 将旧表里的元素复制到新表
- ③ 释放旧表空间

经典示例



1 INSERT

1

2 INSERT

1

2

3 INSERT

1

2

3

4 INSERT

4

5 INSERT

1

2

3

4

5

6 INSERT

6

7 INSERT

7

8 INSERT

8

代价计算



有一个序列包括 n 个插入运算，在最坏情况下一次插入操作的代价为多少

$O(n)$

n 个项的插入操作代价仍为 $O(n)$ ，分析如下：

设 c_i 为第 i 个插入的代价，则

$$c_i = \begin{cases} i & \text{若 } i-1 \text{ 恰为 } 2 \text{ 的幂} \\ 1 & \text{其他} \end{cases}$$

代价计算



i	1	2	3	4	5	6	7	8	9	10
SIZE _i	1	2	4	4	8	8	8	8	16	16
C _i	1	2	3	1	5	1	1	1	9	1

因此，n个插入运算的总代价为 $\sum_{i=1}^n c_i \leq n + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j < n + 2n = 3n$

每个插入运算的平均代价为 $O(n)/n$ ，也就是 $O(1)$ 。

这就是摊还分析的思想，这里采用了聚合分析方法

目录

第一节

Java里面ArrayList例子引入

第二节

什么是摊还分析?

第三节

摊还分析的三种方法

什么是摊还分析?



定义

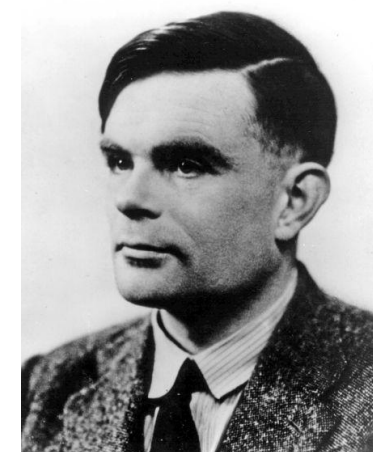
摊还分析里的这种平均也并不是我们平时所熟知的考虑所有情况的平均，而是在最坏情况下，一系列操作的平均成本!

什么是摊还分析?



□ 功过相抵

- 中国：某大臣因办某件事情失败，被皇帝怪罪，看在曾经立下汗马功劳等，希望能够从轻发落。
- 西方：英国政府曾打算对艾伦·图灵（“人工智能之父”）进行严惩，因为他被发现是一个同性恋者，但图灵破译了德国海军密码“谜”，在第二次世界大战中的胜利立下了汗马功劳，可以抵销罪过。



将一生摊开来看，一次失利并不显得怎样罪大恶极。这就是摊还!

摊还分析 VS 平均情况分析



例如，如果打一场篮球赛，

1、平均情况分析

结束后一个队的得分就是这个队所有队员的总得分。用这个得分除以队员数就获得每个队员的平均得分。

2、摊还分析

针对的是最坏情况下每个操作的平均成本。但如果一个球队在打球过程中每个队员都表现的是平时最差的水平。

摊还分析 (特指最坏情况平均) $\neq$ 平均情况分析！

概率分析 VS 摊还分析



1、概率分析:

考虑算法平均执行时间。

考虑同一算法的所有可能输入情况。

如果使用概率，则称为期望运行时间。

2、摊还分析:

操作序列中的平均最坏情况下操作性能/代价。

针对某一数据结构的操作序列。

不使用概率。

什么是摊还分析?



摊还分析 (特指最坏情况平均) $\neq$ 平均情况分析!

摊还分析 (运转 n 次平均) $\neq$ 最坏时间复杂性分析 (理论单次)!

摊还分析 $\neq$ 概率分析

目录

第一节

Java里面ArrayList例子引入

第二节

什么是摊还分析?

第三节

摊还分析的三种方法

摊还分析概述



适用对象： 操作序列中，单个操作的代价波动极大（如：大部分操作 $O(1)$ ，偶尔出现 $O(n)$ 的扩容操作）。

分析目标： 不关注单次操作的最坏情况，而是关注整个操作序列的平均代价。

核心思想： 高代价操作的成本被分摊到了之前的低代价操作上。即使某一步操作很贵，但从长远看，平均下来每一步都很便宜。

摊还分析的三种方法



聚类法、聚合法 (aggregate) 简单, 常用

先求出合计, 然后求平均。先求出操作序列里所有 n 个操作的总代价上界 $T(n)$, 每个操作的摊还代价 $T(n)/n$ 。

记账法、核算法、会计法 (accounting) 复杂

计算每个操作的摊还成本, 然后累加每个个体操作的摊还成本而获得一组操作的总摊还成本。这种摊还分析由于很像会计做账时每笔账务都要清楚记录在案, 最后得出一个总报表(总收益或总亏损)。

摊还分析的三种方法



势能法 (potential) 可靠、准确

与核算法类似，也是分析每个操作的代价，但将势能作为一个整体存储，而与数据结构中的某个对象无关。

三种方法对比

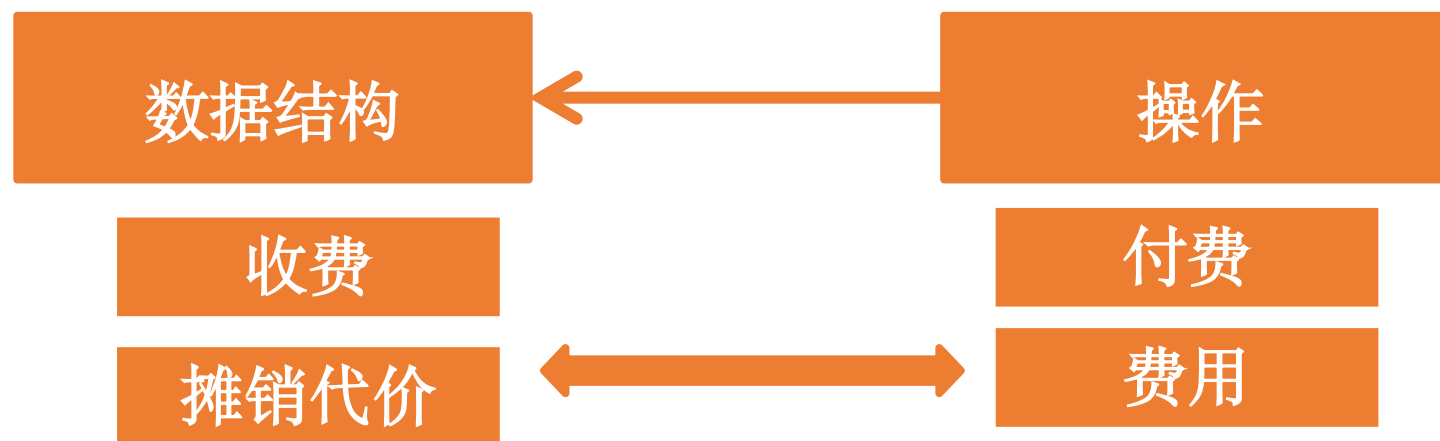


方法	视角隐喻	处理“其他操作代价”的方式	数学表达核心
聚合法	“大锅饭”	事后平摊： 不管谁干的多，最后把总账单加起来除以人数。	$\hat{c}=(\sum c_i)/n$
核算法	“预付费卡”	打包绑定： 在当前操作中，显式地为未来的高代价操作提前买单（存入特定账户）。	$\sum \hat{c}_i \geq \sum c_i$ (信用 ≥ 0)
势能法	“弹簧/电池”	状态内化： 将未来的代价转化为当前的势能储备。操作改变状态，释放或储存能量。	$\hat{c}_i=c_i+\Delta \Phi$



核心思想——费用分配（摊还代价）：

为不同的操作分配不同的费用，每一操作分配到的费用称为该操作的摊还代价，它可视作为数据结构对该操作预收的费用（或理解为该操作对数据结构预付的费用）





□ 超额收费(overcharge)(摊还代价 $>$ 实际成本)

当一个操作的摊还代价大于其实际成本时，数据结构对该操作预收的费用过多，其超额部分作为存款存储在数据结构的某个特定对象上（称为信用）。

□ 收费不足(undercharge)(摊还代价 $<$ 实际成本)

当一个操作的摊还代价小于其实际成本时，数据结构对该操作预收的费用不足，其差额部分可由数据结构的特定对象(是该操作操作的对象)上的存款（信用）支付。



正确选择各操作的摊还代价 (摊还代价的正确性)

要说明每个操作的摊还代价是最坏情况下的平均代价,则必须保证对任意长度 n 的操作序列, 总摊还代价是总的实际代价的一个上界:

$$\forall n, \sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$$

核算法 (17.4节动态表)



回到17.4节动态表

假设收取3元作为对第 i 步操作的费用

1元作为插入操作的费用

2元作为将表翻倍的预存费用，当表的大小增加一倍时，我们就从存款里去取出。1元来移动新项（上次表扩容与这次表扩容之间插入的新项），1元用来移动旧项（上次表扩容之前插入的项目）。

核算法 (17.4节动态表)



在大小为8的表里，插入前4项后表里已经没有元剩下了

0	0	0	0	2	2	2	2
---	---	---	---	---	---	---	---

此时插入第5项，收取3元，1元作为插入的费用，2元存入

继续插入第6项，第7项，第8项

插入第9项时需建立一个新表，此时移动旧表里的8项将存入的8元耗尽，所以前8项变为0

0	0	0	0	0	0	0	0								
---	---	---	---	---	---	---	---	--	--	--	--	--	--	--	--

核算法 (17.4节动态表)



继续插入到第16项

0	0	0	0	0	0	0	0	2	2	2	2	2	2	2	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

到插入第17项时又需建立一个新表，并将旧表元素复制，所以我们始终可以得到一个永不为负的存款余额。

i	1	2	3	4	5	6	7	8	9	10
SIZE _i	1	2	4	4	8	8	8	8	16	16
C _i	1	2	3	1	5	1	1	1	9	1

核算法 (17.4节动态表)



BANK_i代表存款余额。我们在第一项收取2元，对其余每项收取3元。注意，第一项也可以收取3元，这样就会有有多余的1元贯穿在整个过程中，但这并不影响总代价的上限为3n

i	1	2	3	4	5	6	7	8	9	10
$\widehat{C}_i$	2	3	3	3	3	3	3	3	3	3
BANK _i	1	2	2	4	2	4	6	8	2	4

i	1	2	3	4	5	6	7	8	9	10
SIZE _i	1	2	4	4	8	8	8	8	16	16
C _i	1	2	3	1	5	1	1	1	9	1

核算法 (17.4节动态表)



每次插入操作的摊还成本为3，即 $O(1)$ 。

因此， n 次插入操作的摊还成本最坏为 $O(n)$ 。

核算法 (ArrayList)



回到Java ArrayList

收取4元作为对第 i 步操作的费用，用于覆盖当前操作及未来的扩容成本。

费用分配明细：

- 1元（即时消费）：支付当前元素的插入操作成本。
- 3元（预存储蓄）：存入“银行”账户，作为未来扩容时的迁移成本。

注：当数组容量扩充至 1.5 倍时，需从账户取款支付迁移费。

扩容成本拆解（假设从容量 N 扩至 $1.5N$ ）：

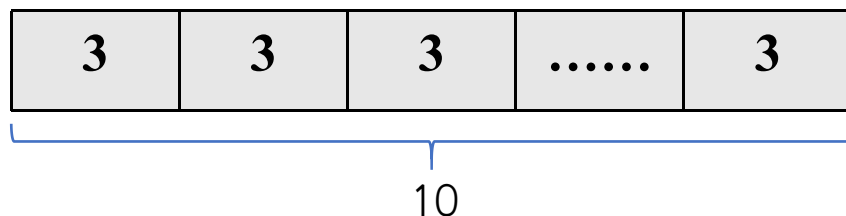
- 新项迁移（1元/个）：移动上次扩容后插入的元素（数量约为 $0.5N$ ）。
- 旧项迁移（2元/个）：移动上次扩容前已存在的元素（数量约为 N ）。

理论依据：由于旧项数量是新项数量的 2 倍，所以预存的 3 元中，1 元给新项，2 元给旧项，刚好平衡。

核算法 (ArrayList)



在插入前10项的时候，账户余下为30元



核算法 (ArrayList)



插入第34项，表扩展到49长度，花费34元，账户剩下 $52+4-34=22$ 元；
之后插入15个，每个带入4元，花掉1元，余下 $22+15*(4-1)=67$ 元；
插入第50项，表扩展到73长度，花费50元，账户剩下 $67+4-50=21$ 元；
之后插入23个，每个带入4元，花掉1元，余下 $21+23*(4-1)=90$ 元；
插入第74项，表扩展到109长度，花费74元，账户剩下 $90+4-74=20$ 元；
之后插入35个，每个带入4元，花掉1元，余下 $20+35*(4-1)=125$ 元；
插入第110项，表扩展到163长度，花费110元，账户剩下 $125+4-110=19$ 元；
.....依次类推

核算法 (ArrayList)



观察趋势：随着操作次数增加，每次扩容后的账户余额呈现缓慢下降趋势

为什么？

计算机中数组长度必须为整数，1.5倍扩容如果多了0.5长度通常采用向下取整，所以每次遇到长度为奇数的ArrayList扩容，账户都会产生 1元 的微小亏空，导致余额持续递减。

额外补丁：当ArrayList满且当前长度N为奇数时，额外收取 1元 补充进账户。

核算法 (ArrayList)



- 每次插入操作的摊还成本为4，仅在奇数扩容时出现微小亏空，通过“动态补丁”补齐，所以实际平均成本略高于 4但非常接近 4。
- 可以直接将单次收费提升至 5元，彻底覆盖奇数扩容的额外成本。此时，账户余额会不断增加。
- 无论取 4 还是 5， $O(4)=O(5)=O(1)$ ，所以单次插入成本是 $O(1)$ 。
- 因此， n 次插入操作的摊还成本最坏为 $O(n)$ 。



□ 与核算法的区别

存款——作为势能保存在整个数据结构上，而非其中的特定的对象上。
。需要时通过释放能量来支付操作的实际代价

□ 势能、势差、势函数、各操作的摊还成本

- 数据结构：D，初态记为 D_0
- 操作：n个，n可变
- 第i个操作的结果： $D_{i-1} \rightarrow D_i$ ， D_{i-1} 和 D_i 表示操作前后D的状态($1 \leq i \leq n$)



□ 势能表示

- 势函数 Φ : 将每个 D_i 映射为一个实数 $\Phi(D_i)(1 \leq i \leq n)$, 函数值看作势能
- 每次摊还的实际成本为 C_i
- 每次摊还的摊还成本为 $\hat{C}_i = C_i + \Phi(D_i) - \Phi(D_{i-1})$
- 操作的摊还代价由两部分构成: 实际成本 C_i 和势差 (操作所引起的势能变化)

$$\begin{cases} \Phi(D_i) - \Phi(D_{i-1}) > 0, & \hat{C}_i \text{ 超额收费, 势能 } \uparrow \\ \Phi(D_i) - \Phi(D_{i-1}) < 0, & \hat{C}_i \text{ 收费不足, 势能 } \downarrow \\ \Phi(D_i) - \Phi(D_{i-1}) = 0, & \hat{C}_i = C_i \end{cases}$$



□ 总的摊还代价及势函数的选择(正确性)

即 i 个操作后，势差为：序列最终的势能和最初的势能之差。此势差 ≥ 0 或 $\Phi(D_i) \geq \Phi(D_0)$ ，则总的摊还代价是总的实际成本的一个上界，但须对任意的 i 成立，故有：

$$\forall i \in I, \Phi(D_i) \geq \Phi(D_0), \text{通常定义 } \Phi(D_0) = 0$$

$$\text{则有: } \forall i \in I, \Phi(D_i) \geq 0$$

势能法



- 因为 $\phi(D_i)$ 项是交叠的，可以消去，所以得出 n 个操作的总摊还代价为

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \phi(D_i) - \phi(D_{i-1})) = \sum_{i=1}^n c_i + \phi(D_n) - \phi(D_0)$$

势能法 (17.4节动态表)



在动态表例子中

我们定义一个势能函数 $\Phi_i = 2 \times num_i - size_i = 2num_i - 2^{\lceil \log i \rceil}$

num_i : 第*i*步操作后的元素个数; $size_i$: 第*i*步操作后的数组容量。

假设 $2^{\lceil \log 0 \rceil} = 0$

满足 $\Phi(D_0) = 0$; $\Phi(D_i) \geq 0 \quad \forall i$

势能法 (17.4节动态表)



现在有8个槽，6个是满的，此时 $i = 6$,

$$\Phi_6 = 2 \times 6 - 2^{\lceil \log 6 \rceil} = 2 \times 6 - 8 = 4$$

势能法—平均代价的计算 (17.4节动态表)



$$\widehat{C}_i = C_i + \phi(D_i) - \phi(D_{i-1})$$

$$= \begin{cases} i & \text{若 } i-1 \text{ 恰为 } 2 \text{ 的幂} \\ 1 & \text{其他} \end{cases} + 2i - 2^{\lceil \log i \rceil} - (2(i-1) - 2^{\lceil \log i-1 \rceil})$$

$$= \begin{cases} i & \text{若 } i-1 \text{ 恰为 } 2 \text{ 的幂} \\ 1 & \text{其他} \end{cases} + 2 - 2^{\lceil \log i \rceil} + 2^{\lceil \log i-1 \rceil}$$

势能法—平均代价的计算 (17.4节动态表)



情形1: $i - 1$ 恰为2的幂时

$$\begin{aligned}\widehat{c}_i &= i + 2 - 2^{\lceil \log i \rceil} + 2^{\lceil \log i \rceil - 1} \\ &= i + 2 - 2(i - 1) + (i - 1) \\ &= i + 2 - 2i + 2 + i - 1 \\ &= 3\end{aligned}$$

势能法—平均代价的计算 (17.4节动态表)



情形2: $i - 1$ 不为2的幂时

$$\begin{aligned}\widehat{C}_i &= 1 + 2 - 2^{\lceil \log i \rceil} + 2^{\lceil \log i \rceil - 1} \\ &= 3\end{aligned}$$

所以我们得到, 每个操作的代价为3, n 个插入操作的平摊代价为 $3n$, 这也是最坏情况下实际代价的上限。所以 n 个操作的代价为 $O(n)$ 。

势能法 (ArrayList)



在ArrayList例子中

我们定义一个势能函数 $\Phi_i = 3 \times num_i - 2 \times size_i = 3 \times num_i - 2 \times (10 \times 1.5^{\lceil \log_{1.5} \frac{i}{10} \rceil})$

num_i : 第*i*步操作后的元素个数; $size_i$: 第*i*步操作后的数组容量。

假设 $1.5^{\lceil \log_{1.5} 0 \rceil} = 0$

满足 $\Phi(D_0) = 0$; $\Phi(D_i) \geq 0 \quad \forall i$

势能法 (ArrayList)



现在有15个槽，12个是满的，此时 $i = 12$,

$$\Phi_{12} = 3 \times 12 - 2 \times 10 \times 1.5^{\lceil \log_{1.5} \frac{12}{10} \rceil} = 36 - 20 = 16$$

势能法—平均代价的计算 (ArrayList)



$$\widehat{C}_i = C_i + \phi(D_i) - \phi(D_{i-1})$$

$$= \begin{cases} i & \text{若 } \frac{i-1}{10} \text{ 恰为 } 1.5 \text{ 的幂} \\ 1 & \text{其他} \end{cases} + 3i - 2 \times 10 \times 1.5^{\lceil \log_{1.5} \frac{i}{10} \rceil} - (3(i-1) - 2 \times 10 \times 1.5^{\lceil \log_{1.5} \frac{i-1}{10} \rceil})$$

$$= \begin{cases} i & \text{若 } \frac{i-1}{10} \text{ 恰为 } 1.5 \text{ 的幂} \\ 1 & \text{其他} \end{cases} + 3 - 20 \times 1.5^{\lceil \log_{1.5} \frac{i}{10} \rceil} + 20 \times 1.5^{\lceil \log_{1.5} \frac{i-1}{10} \rceil}$$

势能法—平均代价的计算 (ArrayList)



情形1: $\frac{i-1}{10}$ 恰为1.5的幂时, $\lceil \log_{1.5} \frac{i}{10} \rceil$ 比 $\lceil \log_{1.5} \frac{i-1}{10} \rceil$ 大1, 于是 $1.5^{\lceil \log_{1.5} \frac{i}{10} \rceil}$ 是 $1.5^{\lceil \log_{1.5} \frac{i-1}{10} \rceil}$ 的1.5倍

$$\begin{aligned}\widehat{C}_i &= i + 3 - 20 \times 1.5^{\lceil \log_{1.5} \frac{i}{10} \rceil} + 20 \times 1.5^{\lceil \log_{1.5} \frac{i-1}{10} \rceil} \\ &= i + 3 - 20 \times 1.5 \times \frac{i-1}{10} + 20 \times \frac{i-1}{10} \\ &= i + 3 - (i - 1) \\ &= 4\end{aligned}$$

势能法—平均代价的计算 (ArrayList)



情形2: $\frac{i-1}{10}$ 不为1.5的幂时

$$\begin{aligned}\widehat{C}_i &= 1 + 3 - 20 \times 1.5^{\lceil \log_{1.5} \frac{i-1}{10} \rceil} + 20 \times 1.5^{\lceil \log_{1.5} \frac{i-1}{10} \rceil} \\ &= 4\end{aligned}$$

所以我们得到，每个操作的代价为4，n个插入操作的平摊代价为4n，这也是最坏情况下实际代价的上限。所以n个操作的代价为O(n)。



湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——